

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)
Question Count: 5

Total Marks: 40

Code of Conduct

I R.Jaideep (192525441) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **R.Jaideep**

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

Experiment 1: Cloud Storage Comparison

Aim

To compare **Object, Block, and File Storage** and select the best model for different workloads.

Storage Comparison

Feature	Object Storage	Block Storage	File Storage
Data organization	Objects	Blocks	Files/directories
Access	HTTP/API	Attached as disk	File protocols
Scalability	Very high	High	High
Performance	Good	Very high	Good
Best for	Backups, images, videos	VM disks, databases	Shared documents
Example	Amazon S3	Amazon EBS	Amazon EFS

1. Object Storage

Object storage stores data as objects along with metadata and a unique identifier.

Advantages:

- Highly scalable
- Low cost for large data
- High durability
- Suitable for unstructured data

Uses: Backups, photos, videos, logs, documents.

2. Block Storage

Block storage divides data into fixed-size blocks and provides them as virtual disks.

Advantages:

- High performance
- Low latency
- Suitable for operating systems and databases

Uses: VM disks, databases, application disks.

3. File Storage

File storage organizes data into files and folders and allows multiple users or systems to access shared files.

Advantages:

- Familiar directory structure
- Easy file sharing
- Suitable for collaborative applications

Uses: Enterprise documents, shared folders, content management.

Selection

- **Backups → Object Storage**
- **VM Disks → Block Storage**
- **Shared Enterprise Documents → File Storage**

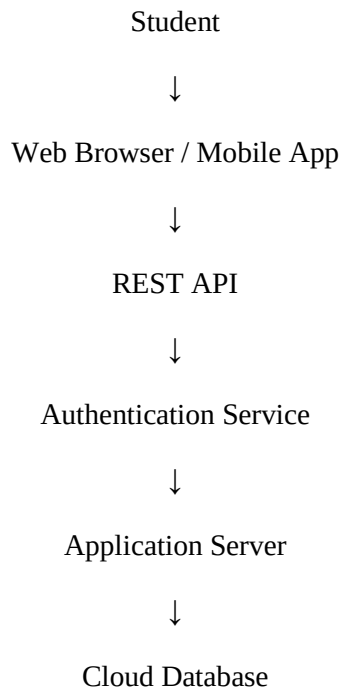
Conclusion

No single storage model is best for every workload. Object storage is ideal for backups, block storage for VM disks, and file storage for shared enterprise documents.

Experiment 2: REST API Workflow for Cloud Student Portal

Aim: To develop a simple REST API workflow for a cloud-based student portal.

Architecture



Example API

Suppose the student wants to view their profile.

Request:

GET /api/students/101

Authorization: Bearer <token>

Authentication

The student first logs in:

POST /api/login

Request:

```
{
  "username": "student101",
  "password": "12345"
}
```

Response:

```
{ "token": "abc123xyz" }
```

The token is then included in subsequent API requests.

Student Data Request

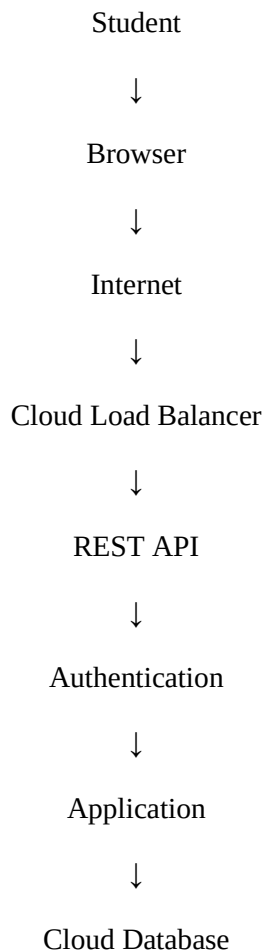
GET /api/students/101

Authorization: Bearer abc123xyz

Response

```
{ "id": 101,  
  "name": "Rahul",  
  "course": "B.Tech AIML",  
  "semester": 2 }
```

Deployment Flow



Conclusion

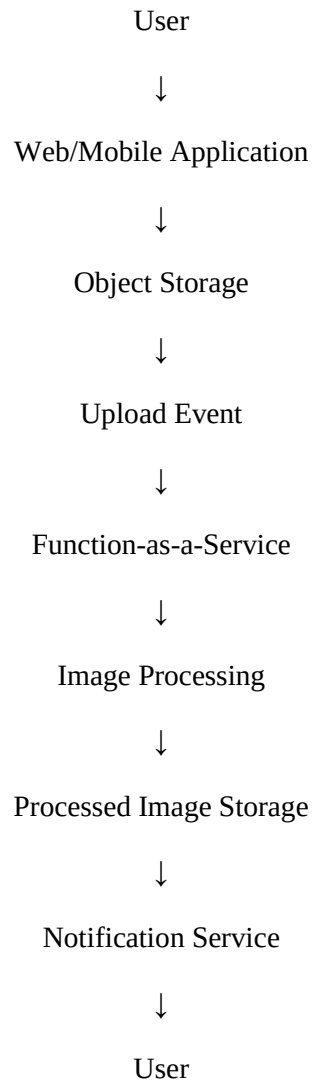
REST APIs allow the student portal to communicate with cloud services using standard HTTP methods such as **GET, POST, PUT, and DELETE**. Authentication protects student information from unauthorized access.

Experiment 3: Serverless Image Upload Processing

Aim

To design a serverless event-driven workflow for processing uploaded images.

Architecture



Working

Step 1:

The user uploads an image through a web or mobile application.

Step 2:

The image is stored in cloud **Object Storage**.

Example:

images/

photo1.jpg

Step 3:

The object storage generates an upload event.

Image Uploaded



Event Trigger

Step 4:

The event triggers a **Function-as-a-Service (FaaS)** function.

Examples:

- AWS Lambda
- Azure Functions
- Google Cloud Functions

Step 5:

The function processes the image.

For example:

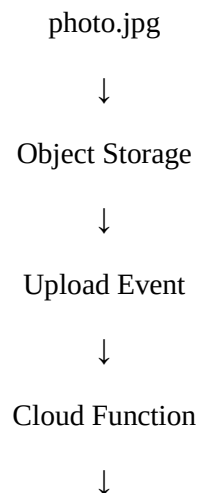
- Resize image
- Compress image
- Generate thumbnail
- Extract metadata

Step 6:

The processed image is stored back in object storage.

Step 7:

A notification service informs the user that processing is complete.

Example

Resize + Compress



thumbnail.jpg



Notification

Advantages

- No server management
- Automatic scaling
- Pay-per-use
- Event-driven processing
- Suitable for unpredictable workloads

Conclusion

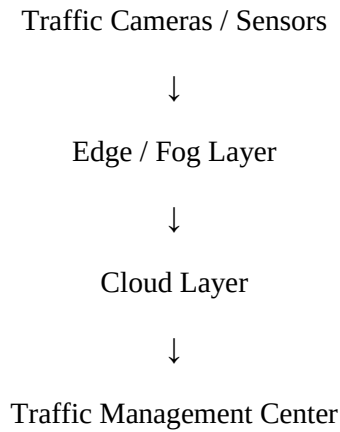
A serverless architecture efficiently processes images without maintaining dedicated servers. Object storage, FaaS, and notification services work together to create an automated workflow.

Experiment 4: Edge vs Fog Computing for Smart Traffic Management

Aim

To compare edge and fog computing for a smart traffic management system.

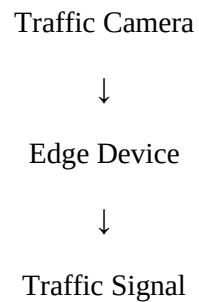
Smart Traffic Architecture



Edge Computing

In edge computing, processing occurs **very close to the data source**.

Example:



The edge device can immediately detect:

- Traffic congestion
- Accidents
- Red-light violations
- Emergency vehicles

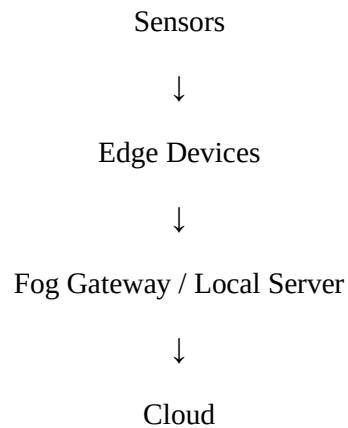
Advantages

- Very low latency
- Fast decision-making
- Reduced bandwidth usage

- Can continue working during poor cloud connectivity

Fog Computing

Fog computing places an intermediate computing layer between edge devices and the cloud.



The fog layer can combine information from multiple intersections.

Comparison

Feature	Edge Computing	Fog Computing
Processing location	At/near device	Between edge and cloud
Latency	Very low	Low
Coverage	Individual device/location	Multiple devices
Bandwidth	Very low cloud traffic	Reduced cloud traffic
Best use	Immediate decisions	Regional coordination

Deployment Location

Edge:

Traffic cameras, traffic lights, roadside sensors.

Fog:

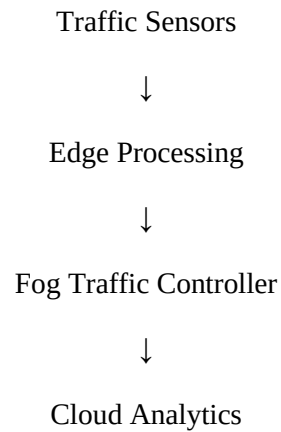
Local traffic control centers or gateways managing several intersections.

Cloud:

Central traffic management system for long-term analysis and storage.

Recommended Architecture

For a smart city, **both edge and fog computing should be used.**



Conclusion

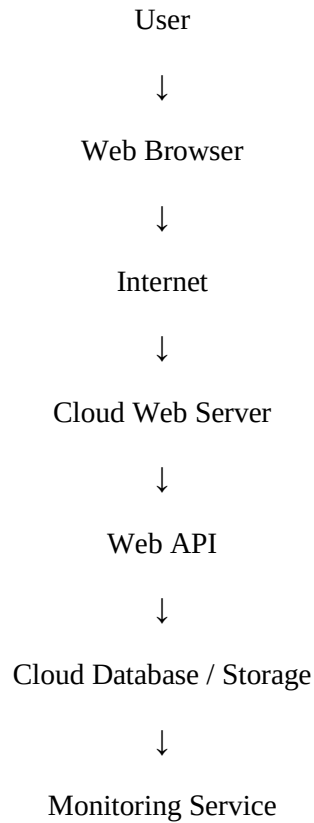
Edge computing is better for **instant traffic decisions**, while fog computing is better for **coordinating multiple nearby devices**. A combined edge-fog-cloud architecture provides low latency and centralized analytics.

Experiment 5: Cloud Application Deployment Workflow

Aim

To demonstrate a cloud application deployment workflow involving client access, browser interaction, web API calls, storage access, and monitoring.

Architecture



Workflow

Step 1: Client Access

The user opens the cloud application using a browser.

User → Browser → Cloud Application

Example:

<https://studentportal.com>

Step 2: Browser Interaction

The browser displays the application's frontend.

The user can:

- Login
- View profile

- Check marks
- Download documents
- Submit assignments

Step 3: Web API Call

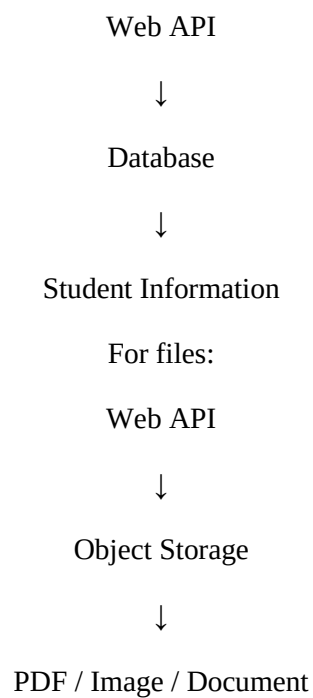
When the user requests information, the browser sends a REST API request.

GET /api/student/101

The API processes the request and communicates with the backend.

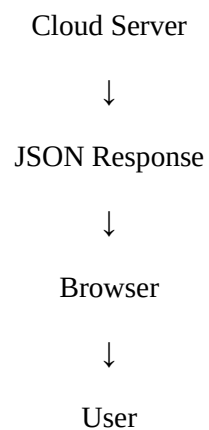
Step 4: Storage Access

The application accesses cloud storage/database.



Step 5: Response

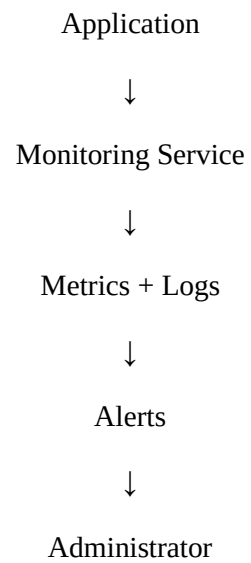
The server sends the requested information back to the browser.



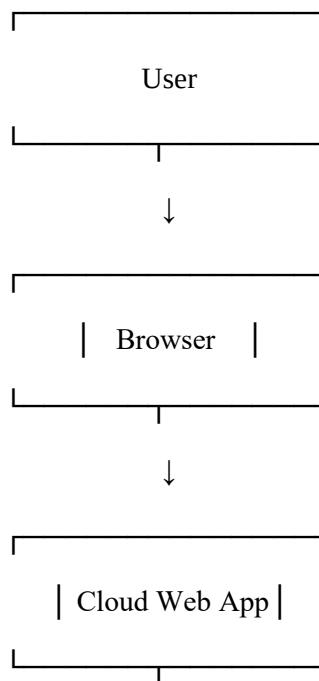
Step 6: Monitoring

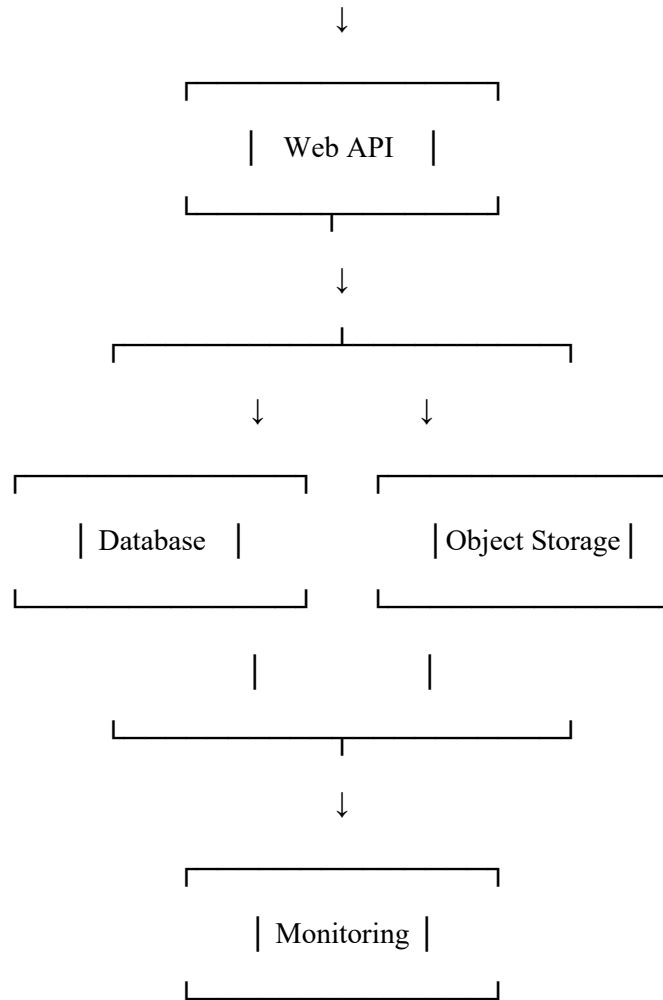
Cloud monitoring tools continuously monitor:

- CPU usage
- Memory usage
- Network traffic
- API response time
- Errors
- Application availability



Complete Deployment Workflow





Conclusion

A cloud application connects the client browser with web APIs, databases, storage, and monitoring services. This architecture provides **scalability, availability, centralized monitoring, and easy access from anywhere.**